



Software Documentation for SnapTray

Table of Contents

- [1. Introduction](#)
- [2. System Overview](#)
- [3. Requirements Specification](#)
- [4. System Architecture](#)
 - [4.1 Components/Modules](#)
 - [4.2 Data Flow](#)
 - [4.3 Technologies](#)
- [5. Design](#)
 - [5.1 Design Principles](#)
 - [5.2 Database Design](#)
 - [5.3 Algorithms and Data Structures](#)
 - [5.4 Security Design](#)
- [6. Implementation](#)
 - [6.1 Directory Structure](#)
 - [6.2 Backend Implementation](#)

- 6.3 Frontend Implementation
- 7. API Reference
- 8. Testing and Validation
- 9. User Manual
- 10. Deployment and Maintenance
- 11. Conclusion and Future Work
- 12. References
- 13. Appendices

1. Introduction

A SnapTray egy webalapú menza-rendelőrendszer, amelynek célja, hogy egyszerűsítse az étkezési rendelések lebonyolítását iskolai környezetben. Fő célja, hogy a diákok, szülők és az étkeztető személyzet közötti interakció gyorsabbá és átláthatóbbá váljon az online rendelés, a valós idejű rendeléskövetés és a biztonságos fizetési lehetőségek révén.

A rendszer három fő felhasználói szerepet szolgál ki: a **diákok** böngészhetnek az étlapok között, rendelhetnek és kezelhetik a virtuális pénztárcájukat; a **szülők** figyelemmel kísérhetik gyermekeik rendeléseit és kezelhetik a fizetéseket; az **adminisztrátorok** számára dashboard biztosít lehetőséget az étlapok kezelésére és statisztikák elemzésére.

A SnapTray modern biztonsági megoldásokat alkalmaz (kétlépcsős azonosítás, email-ellenőrzés, gyakori webes támadások elleni védelem), valamint PayPal és Google Pay integrációt kínál.

2. System Overview

A rendszer egy webalapú rendelési és fizetési platform oktatási intézmények számára, kliens–szerver architektúrában. A backend Node.js alapú Express szerver, a frontend React komponensekből épül fel szerepkör-alapú dashboardokkal. Redis biztosítja a gyorsítótárazást, rate limitinget és az atomi műveleteket, MongoDB az adatok perzisztens tárolását.

Megvalósított fő funkciók

Felhasználókezelés és autentikáció:

- Többszerepkörű felhasználói rendszer (diák, szülő, adminisztrátor)
- Email alapú fiókellenőrzés és kétlépcsős azonosítás (2FA)
- JWT alapú munkamenet-kezelés Redis tárolással
- Jelszó biztonság bcrypt hasheléssel és erősség validációval

Rendelés és menü rendszer:

- Dinamikus menükezelés kategóriákkal és táplálkozási információkkal
- Valós idejű készletkövetés riasztásokkal
- Napi menü funkcionalitás, QR kód integráció, allergén információk

Fizetés feldolgozás:

- PayPal és Google Pay API integráció
- Pénztárca egyenleg rendszer atomi műveletekkel
- Biztonságos tranzakció naplózás és audit trail

3. Requirements Specification

Fő célok

- Biztonságos és ellenőrzött rendelési folyamat
- Digitális pénztárca és hűségpont rendszer
- Skálázható és nagy teljesítményű backend
- Külső fizetési szolgáltatók integrálása (PayPal, Google Pay)

Felhasználókezelés: Regisztráció, email alapú fiókellenőrzés, szerepkör-alapú hozzáférés (diák, szülő, admin).

Hitelesítés és biztonság: JWT alapú hitelesítés, kétlépcsős azonosítás (2FA), rate limiting brute-force védelem ellen.

Rendelések és fizetések: Rendelés leadás, végösszeg számítás, PayPal és Google Pay támogatás, tranzakció naplózás.

Adminisztráció: Felhasználókezelés, statisztikák és riportok.

4. System Architecture

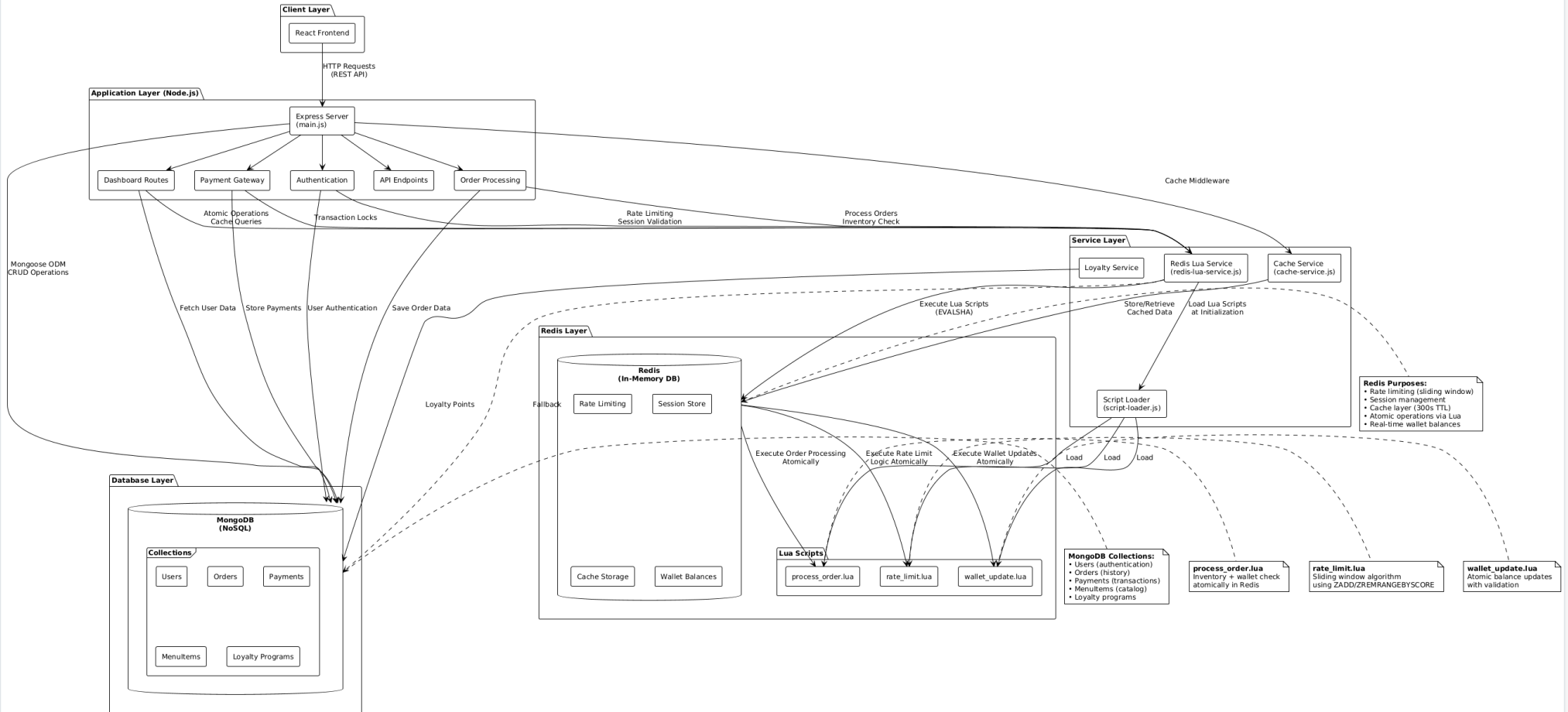
A rendszer három fő rétegből áll:

Megjelenítési réteg (Frontend): React.js komponensek, szerepkör-alapú dashboardok, Tailwind CSS, REST API kommunikáció, valós idejű frissítések Socket.IO-val.

Alkalmazási réteg (Backend): Node.js/Express.js szerver, JWT hitelesítés, üzleti logika szolgáltatásokban, Redis Lua szkriptek atomi műveletekhez, rate limiting middleware.

Adatréteg: MongoDB perzisztens adatbázis Mongoose ODM-mel, Redis cache munkamenetekhez és rate limitinghez.

MERN Stack with Redis + Lua Architecture Repository: bivork/iskolaprojekt



Data Flow:

- Client → Express Server
- Redis cache check first
- Rate limiting via Lua
- Atomic operations in Redis
- Persistent data in MongoDB

Stack:
MongoDB + Express + React + Node.js + Redis + Lua

4.1 Components/Modules

Modul	Leírás
Frontend/UI	React.js, szerepkör-alapú dashboardok, autentikáció, rendelés UI
API Layer	Express.js REST végpontok, üzleti logika
Data Layer	MongoDB (perzisztens), Redis (cache, session)
Auth & Security	JWT, rate limiting, security logging
Payment	PayPal és Google Pay integráció
Loyalty System	Pontszámítás, tier kezelés, kedvezmények

4.2 Data Flow

1. A felhasználó a React frontenddel interaktál, HTTP kéréseket küld.
2. Az Express szerver middleware-eken (hitelesítés, rate limiting, sanitizáció) keresztül irányítja a kéréseket.
3. Az üzleti logika MongoDB-ből vagy Redis cache-ből olvassa az adatokat.
4. Írási műveletek frissítik az adatbázist és érvénytelenítik a cache bejegyzéseket.
5. A válasz JSON formátumban kerül vissza a frontendhez.
6. Valós idejű frissítések Redis pub/sub-on és Socket.IO-n keresztül érkeznek.
7. Minden jelentős esemény naplózásra kerül a SecurityLogs kollekcióba.

4.3 Technologies

Technológia	Indok
Node.js + Express.js	JavaScript full-stack konzisztencia, gyors fejlesztés
MongoDB + Mongoose	Rugalmas dokumentum-séma, skálázható
Redis + Lua	Gyors cache, atomi műveletek, rate limiting
React.js + Tailwind CSS	Komponens-alapú UI, responszív design
JWT + bcrypt	Iparági standard hitelesítés és jelszóvédelem
PayPal / Google Pay	Megbízható, PCI-kompatibilis fizetési integrációk
Socket.IO	Kétirányú valós idejű kommunikáció
Helmet, HPP, CORS	HTTP biztonsági fejlécek és védelmi middleware

5. Design

5.1 Design Principles

Elv	Megvalósítás
Modularitás	Réteges architektúra (routes / services / models), laza csatolás modulok között
Security-First	Defense in depth, least privilege, minden végpont alapértelmezetten hitelesítést igényel

Elv	Megvalósítás
Skálázhatóság	Állapotmentes JWT, Redis cache, MongoDB indexek és connection pooling
Felhasználóközpontú design	Reszponzív Tailwind UI, érthetetlen hibaüzenetek, visszajelzési rendszerek
Megbízhatóság	Graceful degradation, tranzakció-kezelés, átfogó naplózás
Karbantarthatóság	Clean code, git verziókövetés, RESTful API konvenciók, env-alapú konfiguráció
Adatintegritás	Többosztályú validáció, atomi műveletek, audit trail
Platformfüggetlenség	Modern böngészők (Chrome, Firefox, Safari, Edge), mobilreszponzív

5.2 Database Design

5.2.1 Az adatbázis célja

Ez az adatbázis egy iskolai büfé rendszer (MERN stack projekt) részét képezi. Lehetővé teszi a felhasználók számára az étkezés megrendelését, kifizetését és értékelését, valamint támogatja az autentikációt, menükezelést, hűségprogramot, E2EE chatet és biztonsági naplózást. MongoDB-t használ Mongoose ODM-mel, Redis-t a gyorsítótárazáshoz.

5.2.2 Entitások és kapcsolatok

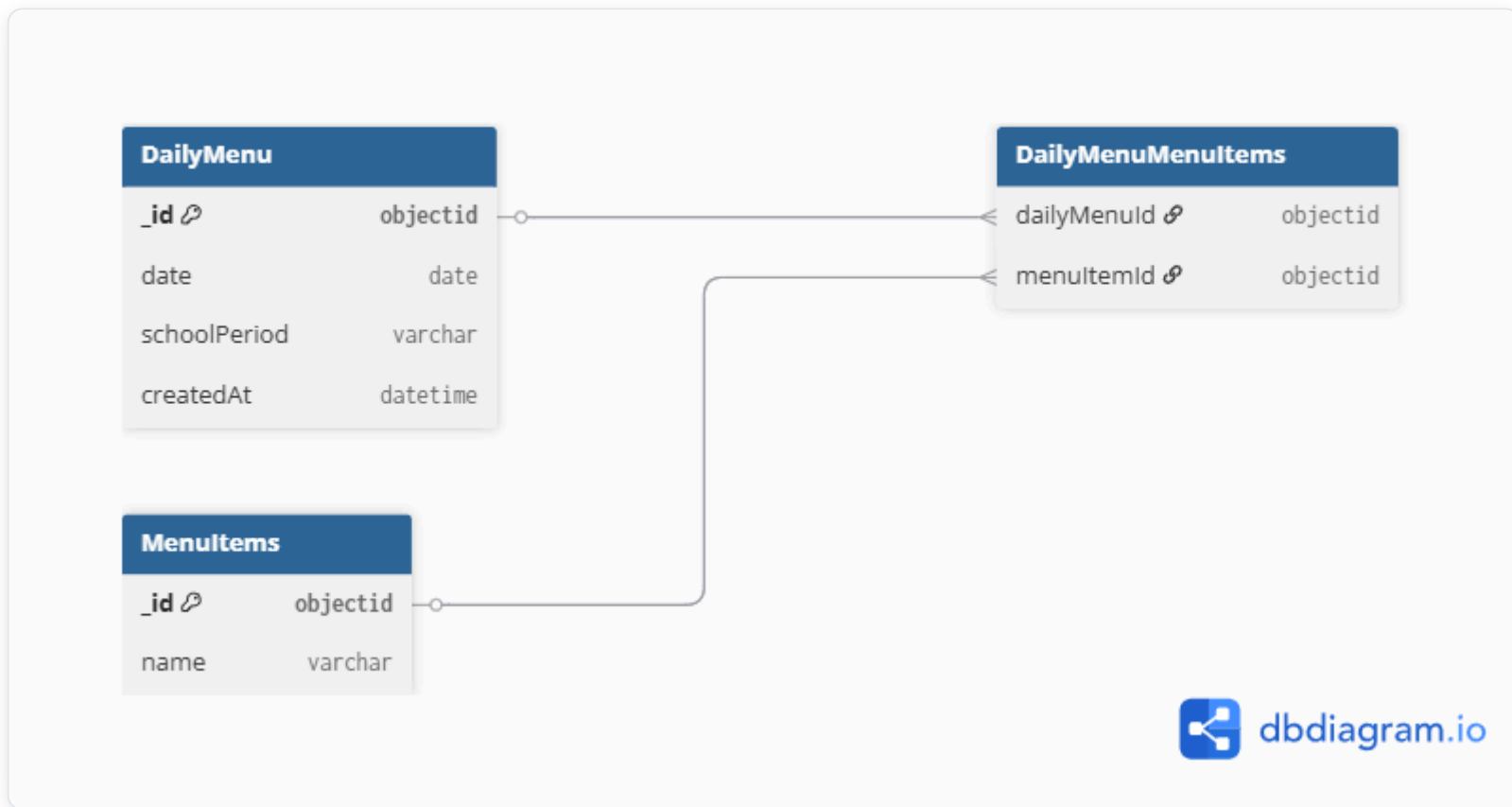
Fő entitások:

Entitás	Leírás
User	Központi entitás, minden máshoz kapcsolódik
MenuItems	Étkezési tételek, beágyazott értékelésekkel

Entitás	Leírás
Order	Felhasználói rendelések, beágyazott OrderItems-szel
Payment	Pénzügyi tranzakciók
DailyMenu	Napi menük (N:M kapcsolat MenuItem-szel)
ParentStudent	Szülő-diák összerendelés
SecurityLogs	Biztonsági események naplója
UserLoyalty	Hűségpontok, tier, kedvezmények
Message	E2EE chat üzenetek (Double Ratchet + X3DH)
PreKey	ECDH előzetes kulcsok az X3DH protokollhoz
StorageBlob	Titkosított session/üzenet történetek (zero-knowledge)
DeviceSyncSession	Ephemerális eszközszinkronizációs munkamenetek (TTL index)

Kapcsolatok:

- **User** 1:N → Payment, Order, SecurityLogs, UserLoyalty, Message, PreKey, StorageBlob, ParentStudent
- **MenuItem** 1:N → OrderItems (beágyazva az Order-be), Review (beágyazva)
- **DailyMenu** N:M ↔ MenuItem (linking table)



5.2.3 E2EE Chat az adatbázisban

A chat rendszer Double Ratchet protokollra és X3DH kulcscserére épül. A szerver nem fér hozzá az üzenetek tartalmához — csak a szükséges metaadatokat tárolja. Az üzenetek és kulcsok gyorsan cserélődnek; a PreKey és DeviceSyncSession kollekciók magas churn-t kezelnek TTL indexekkel.

5.3 Algorithms and Data Structures

5.3.1 Data Structures

- **MongoDB Collections:** BSON dokumentumok, beágyazott dokumentumokkal (OrderItems az Order-ben), ObjectId referenciákkal és tömbökkel (allergének, napi menü tételek).
- **Redis Strings/Hashes:** Session tárolás, cached user adatok, komplex dashboard objektumok.
- **Redis Sorted Sets:** Sliding window rate limiting, timestamp alapú score-okkal.
- **JavaScript:** Objektumok API válaszokhoz, tömbök cart és batch műveletekhez, Map/Set cache lookupokhoz.

5.3.2 Core Algorithms

Password Hashing (bcrypt):

```
const hashedPassword = await bcrypt.hash(password, 12);  
const isValid = await bcrypt.compare(password, hashedPassword);
```

JWT Token Generation (HS256):

```
const token = jwt.sign(payload, secretKey, { expiresIn: '24h' });  
const decoded = jwt.verify(token, secretKey);
```

IP Hashing (SHA-256, GDPR compliance):

```
const hashedIP = crypto.createHash('sha256').update(ipAddress).digest('hex');
```

Loyalty Point Calculation:

```
function ConvertPoints(dollarAmount, tier, healthLevel, date) {
  let total = 0;
  for (let i = 0; i < Math.floor(dollarAmount); i++) {
    total += Math.floor(Math.random() * 6) + 4; // 4-9 points per dollar
  }
  if (isHoliday(date)) total *= 1.5;
  else if (isHolidaySeason(date)) total *= 1.2;
  total *= 1 + (healthLevel * 0.2);
  const tierBonus = DISCOUNT_RATES[tier] || 0;
  total *= 1 + tierBonus;
  return Math.floor(total);
}
```

Tier Determination:

```
const determineTier = (totalPoints) => {
  if (totalPoints >= 20000) return 'PLATINUM';
  if (totalPoints >= 8000) return 'GOLD';
  if (totalPoints >= 2500) return 'SILVER';
  if (totalPoints >= 1200) return 'BRONZE';
  return 'NONE';
};
```

Menu Item Filtering:

```
const filterMenuItems = (items, filters) =>
  items.filter(item =>
    (!filters.category || item.category === filters.category) &&
    (!filters.priceRange || isInRange(item.price, filters.priceRange)) &&
    (!filters.allergens || !hasAllergens(item, filters.allergens)) &&
    (!filters.searchTerm || item.name.toLowerCase().includes(filters.searchTerm.toLowerCase()))
  );
```

Sliding Window Rate Limiting (Redis Lua):

```
local key = KEYS[1]
local window = tonumber(ARGV[1])
local max_requests = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local current_count = redis.call('ZCARD', key)

if current_count >= max_requests then
    return {0, current_count}
end

redis.call('ZADD', key, now, now)
redis.call('EXPIRE', key, window)
return {1, current_count + 1}
```

Atomic Wallet Update (Redis Lua):

```
local current_balance = tonumber(redis.call('GET', KEYS[1]) or '0')
local new_balance = current_balance + tonumber(ARGV[1])
if new_balance < 0 then
    return redis.error_reply('INSUFFICIENT_FUNDS')
end
redis.call('SET', KEYS[1], new_balance)
return new_balance
```

reCAPTCHA Verification:

```
const verifyRecaptcha = async (token) => {
  const result = await fetch('https://www.google.com/recaptcha/api/siteverify', {
    method: 'POST',
    body: new URLSearchParams({ secret: process.env.RECAPTCHA_SECRET, response: token })
  }).then(r => r.json());
  return result.score >= 0.5; // 0.0 = bot, 1.0 = human
};
```

5.3.3 Performance Optimization

- **Database indexes:** Compound indexes on frequently queried fields (e.g., `{ email: 1, isVerified: 1 }`, `{ userId: 1, orderDate: -1 }`).
- **Pagination:** Skip-limit with total count for large datasets.
- **Batch processing:** `bulkWrite` for multi-record updates.
- **Cache middleware:** Redis cache-first, fallback to MongoDB, with pattern-based invalidation on writes.

5.4 Security Design

5.4.1 Security Features

Feature	Details
Authentication	JWT (HS256), role-based access control (RBAC)
Password storage	bcrypt, 10–12 salt rounds
2FA	Implemented; full integration in progress
Rate limiting	<code>express-rate-limit</code> (general) + Redis Lua sliding window (admin/dashboard)
Input validation	Client-side, server-side, database-level; <code>express-validator</code> , Mongoose schemas

Feature	Details
XSS / injection	<code>xss-clean</code> , <code>helmet</code> , <code>express-mongo-sanitize</code>
CSRF	Partial — tokens planned for all state-changing operations
CORS	Strict policy; only official frontend domain allowed
Security headers	Helmet.js (CSP, HSTS, etc.), no eval(), nonce-based inline scripts
IP hashing	SHA-256 before storage in SecurityLogs (GDPR Art. 32)
reCAPTCHA	Google reCAPTCHA v3 on registration and login
Geolocation	iplocate.io for VPN/Proxy/Tor detection and risk scoring
Payment security	PayPal/Google Pay PCI-compliant gateways
Environment secrets	All credentials in <code>.env</code> , never in version control

5.4.2 Security Testing

- Automated test suites for authentication flows, input sanitization, and rate limiting.
 - Periodic manual penetration testing for vulnerabilities not caught by automated tools.
-

6. Implementation

6.1 Directory Structure

```
├─ config/
│   ├── DATABASE_CONSTANTS.JS
│   ├── database_queries.js
│   └─ hu.json
├─ data/
│   ├── disposable_email_list.json
│   ├── Most_used_passwords.json
│   └─ database_test/
├─ docs/
├─ public/
│   ├── chat/
│   ├── dashboard/
│   │   ├── admin/
│   │   ├── editor/
│   │   ├── parent/
│   │   └─ student/
│   └─ Order/
├─ src/
│   ├── main.js
│   ├── auth/
│   │   ├── 2fa.js
│   │   ├── login.js
│   │   ├── register.js
│   │   ├── password_reset.js
│   │   └─ validation.js
│   ├── cache/
│   │   ├── ChangeStreamManager.js
│   │   └─ KeyRegistry.js
│   ├── dashboard/
│   ├── LoyaltySystem/
│   │   └─ loyalty-service.js
│   ├── models/
│   ├── Orders/
│   └─ payments/
```

```
| └─ services/  
└─ tests/  
    └─ performance_tests/
```

6.2 Backend Implementation

6.2.1 Technology Stack

Node.js + Express.js, MongoDB + Mongoose, Redis + Lua scripting, JWT, bcrypt, PayPal & Google Pay APIs, [Socket.IO](#).

6.2.2 Main Application Structure

- **src/main.js** : Entry point — sets up Express, middleware (Helmet, CORS, session, rate limiting), and mounts all routers.
- **Routers**: Modularized by domain (`auth` , `dashboard` , `orders` , `payments` , `chat`).
- **Models**: Mongoose schemas in `src/models/` .
- **Services**: Business logic isolated in service modules (`loyalty-service.js` , `paypal-service.js` , `cache-service.js` , etc.).

6.2.3 Authentication & Security

Registration validates input, verifies reCAPTCHA, hashes the password with bcrypt, sends an email verification code, and logs the event.

Login verifies credentials, issues a JWT, logs IP/location risk, and applies rate limiting per IP.

```
// Registration snippet - src/auth/register.js  
const hashedPassword = await bcrypt.hash(password, 10);  
const user = new User({ username, password: hashedPassword, email });  
await user.save();  
await createSecurityLog('USER_REGISTER', { username, email }, clientId);  
res.status(200).json({ message: 'Registration successful! Please check your email for verification.' });
```

6.2.4 Order & Payment Processing

The order flow validates the cart against live stock, creates a pending order in MongoDB, calls the PayPal or Google Pay API, then on capture: confirms the payment, deducts stock, awards loyalty points, and logs the transaction.

```
// Order creation – src/api.js
router.post('/orders', async (req, res) => {
  // validate user & stock...
  const { jsonResponse, httpStatusCode } = await paypalService.createOrder(cart, currency, amount);
  // save order to DB...
  res.status(httpStatusCode).json(jsonResponse);
});
```

6.2.5 Caching & Performance

Redis is used for caching menu items, sessions, and dashboard data. A `cacheResult` middleware wraps endpoints to serve cached responses when available, with MongoDB change streams used to invalidate stale cache entries atomically.

```
function cacheResult(cacheKey, ttl = 300) {
  return async (req, res, next) => {
    if (!isRedisAvailable()) return next();
    const key = typeof cacheKey === 'function' ? cacheKey(req) : cacheKey;
    const cached = await redisClient.get(key);
    if (cached) return res.status(200).json(JSON.parse(cached));
    const originalJson = res.json;
    res.json = function(data) {
      redisClient.setEx(key, ttl, JSON.stringify(data));
      return originalJson.call(this, data);
    };
    next();
  };
}
```

6.2.6 Loyalty System

Points are calculated per order using a random value (4–9 points per dollar), multiplied by holiday, health score, and tier bonuses. Tiers: NONE → BRONZE (1200 pts) → SILVER (2500) → GOLD (8000) → PLATINUM (20000). See `src/LoyaltySystem/loyalty-service.js` and `config/DATABASE_CONSTANTS.JS` for rates.

6.2.7 Rate Limiting

Two strategies are used: `express-rate-limit` with a Redis store for general API routes, and a custom Redis Lua sliding window script for admin/dashboard routes (30 req/min). The Lua implementation is atomic — it runs as a single uninterruptible transaction, preventing race conditions under high concurrency. See the Lua script in section 5.3.2.

6.2.8 Extensibility & Maintainability

The backend uses a layered, service-oriented architecture (routes → services → models). Configuration is environment-based via `.env`. Error handling is centralized through Express error middleware, which logs security events and returns safe messages to clients. The stateless JWT design and Redis session storage allow horizontal scaling. Dependencies are managed with `npm audit` and kept up to date.

6.3 Frontend Implementation

(Section to be completed)

7. API Reference

All endpoints require an active session unless marked as **public**. Authentication failures return `401 Unauthorized`; insufficient permissions return `403 Forbidden`. Admin-only routes are highlighted in **red**.

Main Application Routes

Method	Endpoint	Auth	Description
GET	/login	Public	Login page
GET	/register	Public	Registration page
GET	/password-reset/:token	Public	Password reset page
GET	/pay	Session	Payment page
GET	/chat	Session	E2EE chat UI

Authentication Routes

Method	Endpoint	Auth	Description
POST	/register	Public	User registration
POST	/login	Public	User login
POST	/logout	Session	User logout
GET	/logout	Session	Logout (redirect)
POST	/2fa	Public	Two-factor authentication
POST	/email-verification/verify-code	Public	Verify email code
GET	/email-verification/verify/:token	Public	Verify email via token
POST	/password-reset/	Public	Request password reset

Method	Endpoint	Auth	Description
GET	/password-reset/:token	Public	Password reset form
POST	/password-reset/:token	Public	Submit new password
POST	/forgot-password/	Public	Forgot password request

Example — POST /register:

```
// Request
{ "username": "johndoe", "password": "SecurePass123!", "email": "john@example.com", "isParent": "false", "g-recaptcha-response": "token" }
// Response 200
{ "message": "Registration successful! Check your email for verification code." }
// Errors: 400 (validation/CAPTCHA), 429 (rate limited), 500
```

Example — POST /login:

```
// Request
{ "username": "johndoe", "password": "SecurePass123!" }
// Response 200: "Welcome, johndoe"
// Errors: 400, 401 (invalid credentials), 429, 500
```

Dashboard Routes

Method	Endpoint	Auth	Description
GET	/dashboard/	Session	Main dashboard (role redirect)
GET	/dashboard/admin	Admin	Admin dashboard

Method	Endpoint	Auth	Description
GET	/dashboard/student	Student/Parent	Student dashboard

Admin Dashboard API Routes

All routes require admin session. Errors: 401 , 403 , 500 .

Method	Endpoint	Description
GET	/dashboard/admin/usercount	Total user count
GET	/dashboard/admin/userlist	List of all users
GET	/dashboard/admin/stats	System statistics
GET	/dashboard/admin/signup-stats	Signup statistics
GET	/dashboard/admin/orders	All orders
GET	/dashboard/admin/soldout	Sold-out items
GET	/dashboard/admin/itemcount	Menu item count
GET	/dashboard/admin/menulist	Menu item list
GET	/dashboard/admin/stockalerts	Low-stock alerts
GET	/dashboard/admin/paymentstats	Payment statistics
GET	/dashboard/admin/health	System health check
GET	/dashboard/admin/menuitem_export	Export menu items

Method	Endpoint	Description
GET	/dashboard/admin/delete_menuitem/:id	Delete menu item
POST	/dashboard/admin/create_menuitem	Create menu item
PUT	/dashboard/admin/menuitem/:id	Update menu item

Example — GET /dashboard/admin/health response:

```
{ "overall": "ok", "services": { "database": "healthy", "redis": "healthy", "sessions": "healthy", "externalServices": { "paypal": "configured", "googlepay": "configu
```

Student Dashboard Routes

Method	Endpoint	Auth	Description
GET	/dashboard/student/freeze_account	Student	Freeze account page
POST	/dashboard/student/parent/link	Student	Link parent account

Order Management Routes

Method	Endpoint	Auth	Description
GET	/Order/	Session	Order page
GET	/Order/menu_items	Session	Available menu items
GET	/Order/:orderId	Session	Order details
POST	/Order/Order	Session	Create new order

Method	Endpoint	Auth	Description
PUT	/Order/:orderID/status	Session	Update order status
POST	/Order/:orderID/capture	Session	Capture payment

Example — POST /Order/Order:

```
// Request
{ "cart": [{ "id": "item_id", "quantity": 2, "price": 8.99 }], "currency": "USD", "amount": 17.98 }
// Response: PayPal order JSON
// Errors: 400 (invalid cart/stock), 401, 500
```

General API Routes

Method	Endpoint	Auth	Description
GET	/api/test	Public	API health check
GET	/api/current_user	Session	Logged-in user info
GET	/api/menu-items	Session	Available menu items
POST	/api/orders	Session	Create PayPal order
POST	/api/orders/:orderID/capture	Session	Capture PayPal payment
POST	/api/orders/googlepay	Session	Create Google Pay order
POST	/api/orders/googlepay/complete	Session	Complete Google Pay transaction
POST	/api/payments/paypal	Session	PayPal payment processing

Method	Endpoint	Auth	Description
POST	/api/payments/googlepay	Session	Google Pay payment processing

Example — POST /api/orders/googlepay/complete:

```
// Request
{ "orderId": "order_123", "paymentMethodData": {}, "transactionId": "txn_456" }
// Response
{ "success": true, "orderId": "order_123", "transactionId": "txn_456", "loyaltyPointsAwarded": 8 }
// Errors: 400, 401, 404, 500
```

Chat API & WebSocket Routes

Method	Endpoint	Auth	Description
GET	/chat	Session	Chat UI
WS	/chat	Session	WebSocket for real-time chat
POST	/chat/setup-e2ee	Session	Set up E2EE public key
GET	/chat/public-key/:userId	Session	Get user's public key
POST	/chat/send-message	Session	Send encrypted message
GET	/chat/messages/:otherUserId	Session	Get conversation messages
GET	/chat/message/:messageId	Session	Fetch single message
POST	/chat/message/:messageId/replace	Session	Mark message as replaced
PUT	/chat/messages/read/:otherUserId	Session	Mark messages as read

Method	Endpoint	Auth	Description
GET	/chat/conversations	Session	Conversation list
GET	/chat/search-users	Session	Search users
GET	/chat/e2ee-status	Session	E2EE status for current user
POST	/chat/reset-e2ee	Session	Reset E2EE keys
POST	/chat/backup-keys	Session	Backup encrypted private key
GET	/chat/has-key-backup	Session	Check for key backup
GET	/chat/restore-keys	Session	Restore private key from backup
POST	/chat/admin/clear-all-e2ee	Admin	Clear all E2EE data (admin only)
POST	/chat/request-sender-recovery	Session	Request sender-key recovery
GET	/chat/pending-recovery	Session	Get messages needing recovery

WebSocket events: `newMessage`, `messageReplaced`, `processPendingRecovery`

All chat messages are encrypted client-side (E2EE). The server stores only ciphertext and metadata.

Backend Models (MongoDB)

User Model

Field	Type	Notes
username	String	Required, unique

Field	Type	Notes
password	String	bcrypt hashed (10 rounds)
email	String	Required, unique, validated
isVerified	Boolean	Default false
usertype	Enum	admin / student / parent / teacher / frozen / editor
balance	Number	Wallet balance, default 0
isBanned	Boolean	Default false
identity	Subdocument	E2EE identity fields
devices	Array	Registered device info

Caching — Redis Key Registry

The project uses a centralized `src/cache/KeyRegistry.js` to map MongoDB collections to their affected Redis cache keys, ensuring consistent cache invalidation on writes:

```
const keyRegistry = {
  users:      (userId) => [`user:username:${userId}`, 'admin:usercount', 'admin:userlist', ...],
  menuitems:  (itemName) => [`menu_item:${itemName}`, 'admin:menulist', 'editor:menulist', ...],
  orders:     (orderId) => [`order:${orderId}`, 'admin:orders', 'admin:most_bought_items_alltime', ...],
  payments:   (userId) => [`student:transactions:${userId}`, 'admin:paymentstats', ...],
  userloyalties: (userId) => [`student:loyalty:${userId}`, 'wallet:user:${userId}', ...],
  // ... (see KeyRegistry.js for full list)
};
```

8. Testing and Validation

(Section to be completed)

9. User Manual

(Section to be completed)

10. Deployment and Maintenance

(Section to be completed)

11. Conclusion and Future Work

(Section to be completed)

12. References

(Section to be completed)

13. Appendices

(Section to be completed)